

Mutation Analysis

Guillermo Polito

guillermo.polito@inria.fr
@guillep



inria



**Université
de Lille**

Goals

- Introduce Mutation Analysis as a technique to assess test quality
- The competent developer hypothesis
- Computing mutation scores

Automated Tests

SetTest >> testAdd

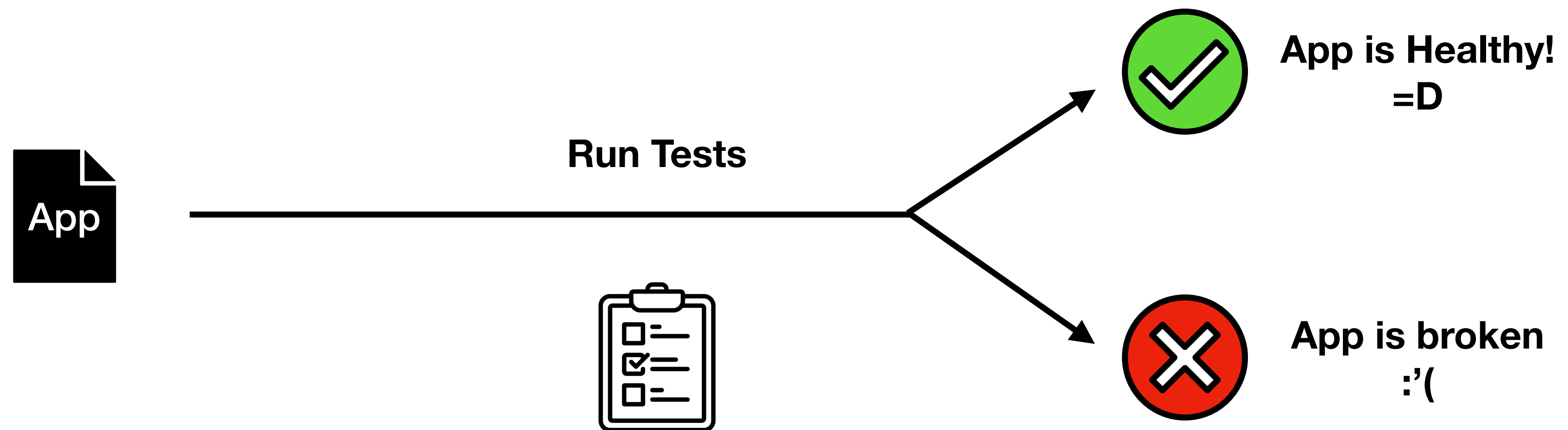
```
| aSet |  
"Context"  
aSet := Set new.
```

```
"Stimuli"  
aSet add: 5.  
aSet add: 5.
```

```
"Check"  
self assert: aSet size equals: 1.
```

in this context
when this happens
then this should happen

We love tests



What is a good test?

“A good test is a test that catches bugs”

- me

**WHO
WATCHES
THE**

TESTS

How do we know if a test catches bugs?

How do we know if a test catches bugs?

- Introduce artificial bugs
- See if the test suite detects them
- What kind of bugs?

Competent developer hypothesis

- Developers are capable people
- Mistakes/bugs are small details easily overseen. E.g.,
 - Missing +/- 1 in a loop
 - An inverted conditional
 - Signed/unsigned

Thousands of simple mutations

- Control flow bugs
- Arithmetic bugs
- Logic bugs
- Overflow bugs
- Typos

Inspector on 3090 mutants, 1599 killed, 1491 alive...

a MTGeneralResult (3090 mut...

Surviving Mutants Killed Mutants Terminated Mutants Excluded Tests Raw Breakpoints Meta

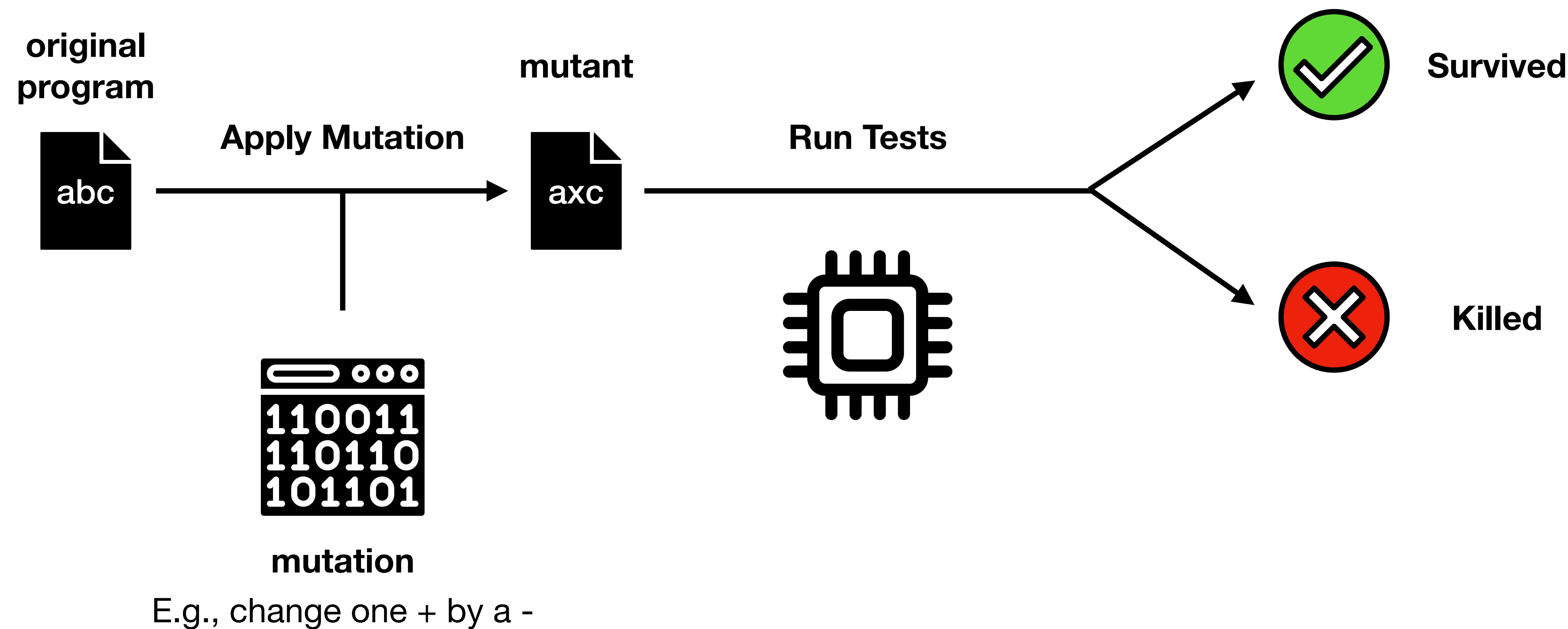
Results

- 1 Replace #ifTrue: receiver with false in MicAnchorBlock>>#printOn:
- 2 Replace detect: block with [:each | true] when #detect:ifNone: in Class>>#documentExampleCode
- 3 Replace #ifFalse: receiver with false in MicInlineDelimiter class>>#initializeDelimiters
- 4 Replace #ifTrue:ifFalse: receiver with false in MicMicrodownTextualBuilder>>#mathblock:firstLineAssociations:withCaption:
- 5 Decrease a literal integer in Package>>#buildMicroDownUsing:withComment:
- 6 Replace #ifTrue: with #ifFalse: in BaselineOf class>>#buildMicroDownUsing:withComment:
- 7 Replace #= with #~=' in MicInlineDelimiter class>>#initializeDelimiters
- 8 Replace #or: with #and: in MicFileResourceReference>>#loadChildren

```
mathblock: aString firstLineAssociations: aCol withCaption: aCaption
{
  self raw: MathOpeningBlockMarkup.
  aCol
    do: [ :each |
      self
        raw: each key;
        raw: '='.
      each key = #caption
        ifTrue: [ aCaptionBlock value ]
        ifFalse: [ "so that we can put format such as bold"
          self raw: each value value ] ]
    separatedBy: [ self raw: '&' ].
  self newline
}
```

```
mathblock: aString firstLineAssociations: aCol withCaption: aCaption
{
  self raw: MathOpeningBlockMarkup.
  aCol
    do: [ :each |
      self
        raw: each key;
        raw: '='.
      false
        ifTrue: [ aCaptionBlock value ]
        ifFalse: [ "so that we can put format such as bold"
          self raw: each value value ] ]
    separatedBy: [ self raw: '&' ].
  self newline
}
```

Mutation Analysis



The insight

- Survived mutants were either

1. not covered

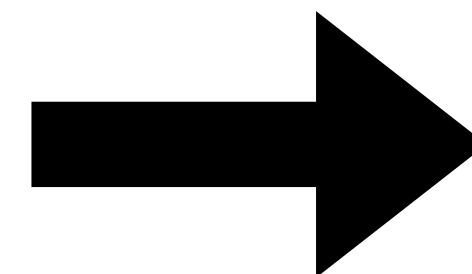
2. not asserted



improve your tests!!

3. or semantically **equivalent**.

E.g. $A+B=A-B$ if $B=0$ always



bias our results

Mutation Score

- Run each mutation independently

- Score:

$$\frac{\text{\#Killed}}{\text{\#Mutants}}$$

Problems of Mutation Analysis

$$\text{Runtime} = \text{Time}(\text{tests}) * \text{\#Mutants}$$

Optimizing of Mutation Analysis

$$\text{Runtime} = \text{Time}(\text{tests}) * \# \text{Mutants}$$

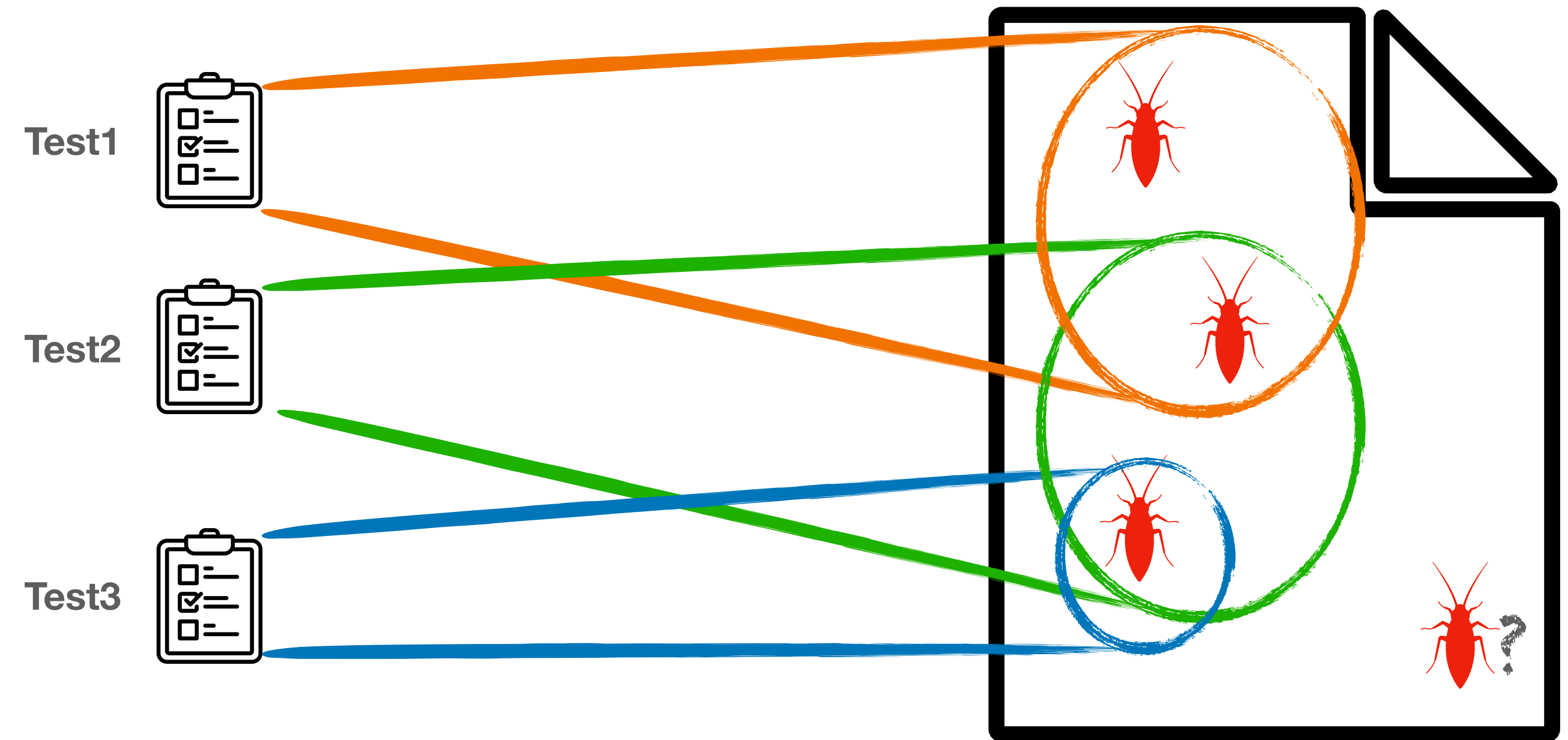
Can we run **less tests**?

Can we run **less mutants**?

Ideally... no impact on score!

Selecting Mutants and Tests from Coverage

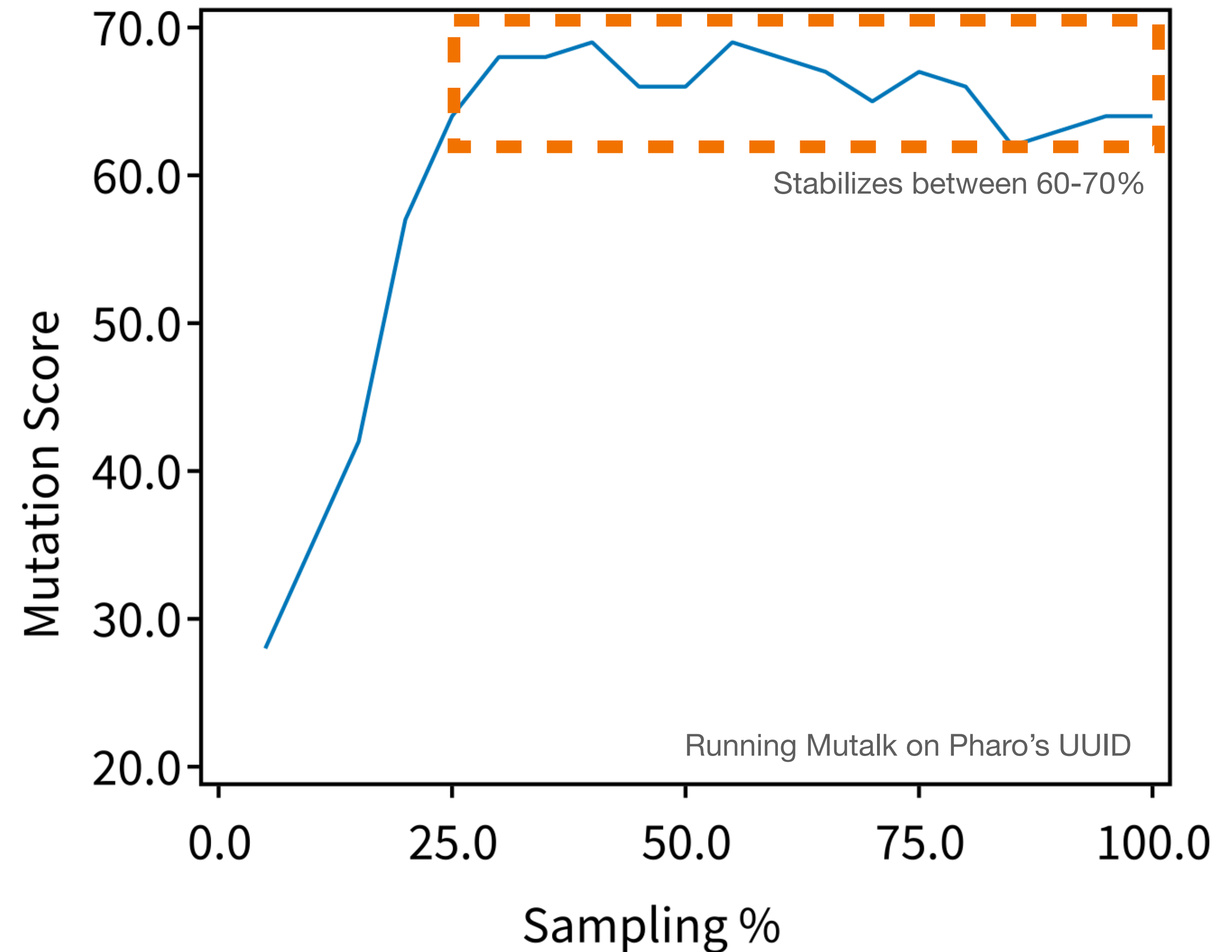
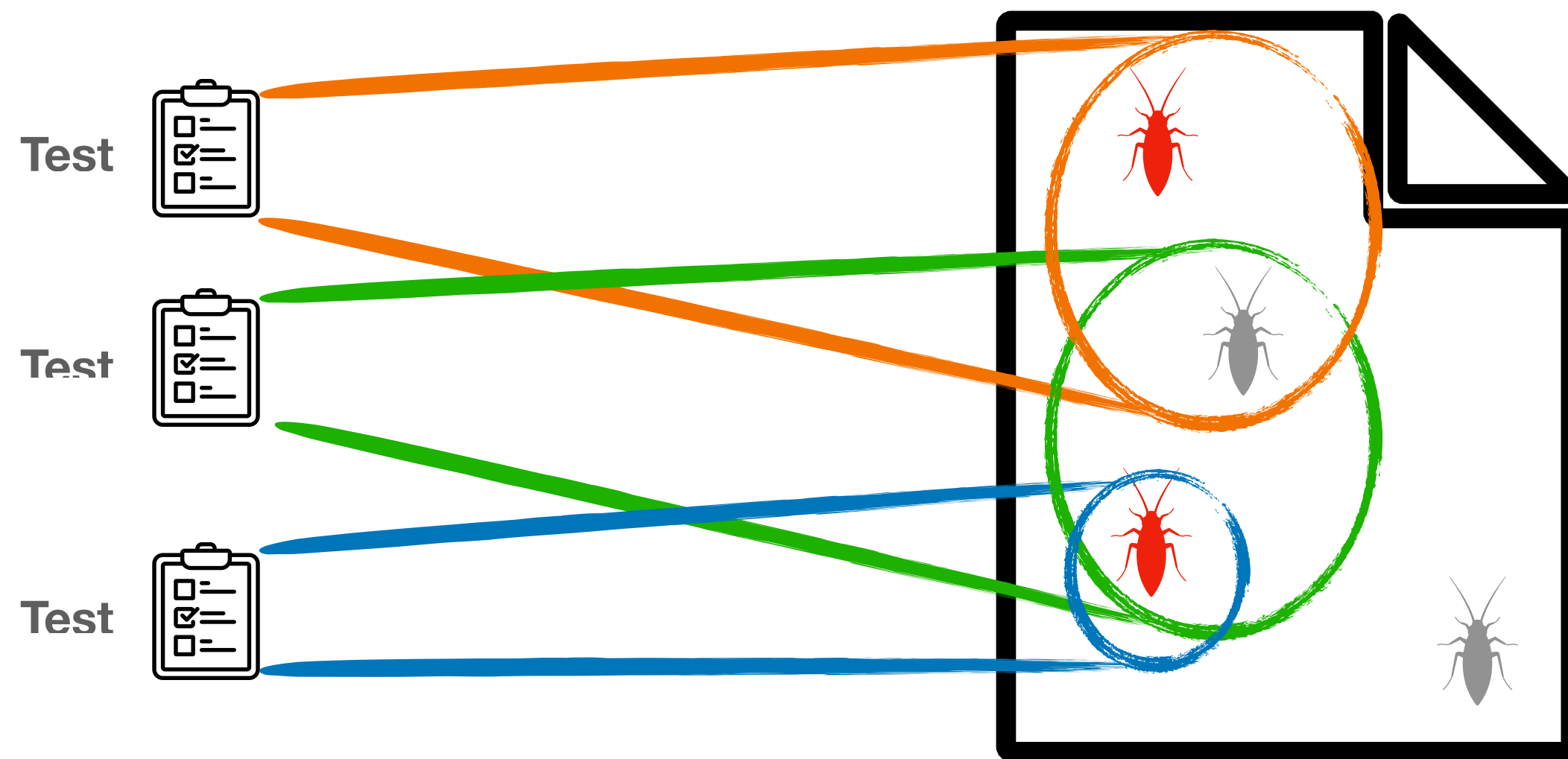
- Run *only* tests covering mutants
- Only mutate covered code



- What if lots of mutants remain?

Random Sampling of Mutants

- Lots on research on it
- Aggressively reduces run time

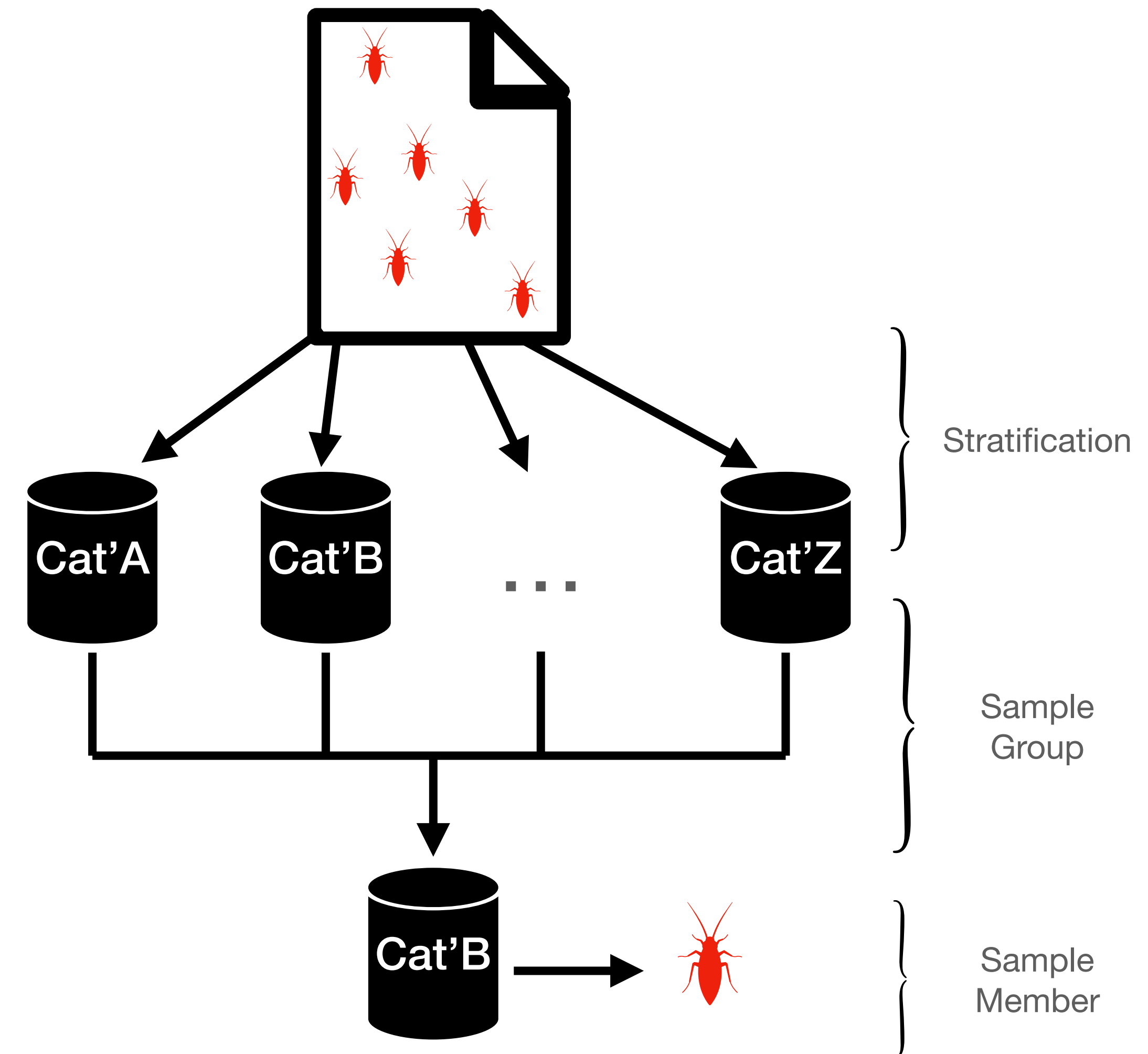


Guiding Random Sampling of Mutants

- Stratified sampling with different strategies
 - per class, method, ...

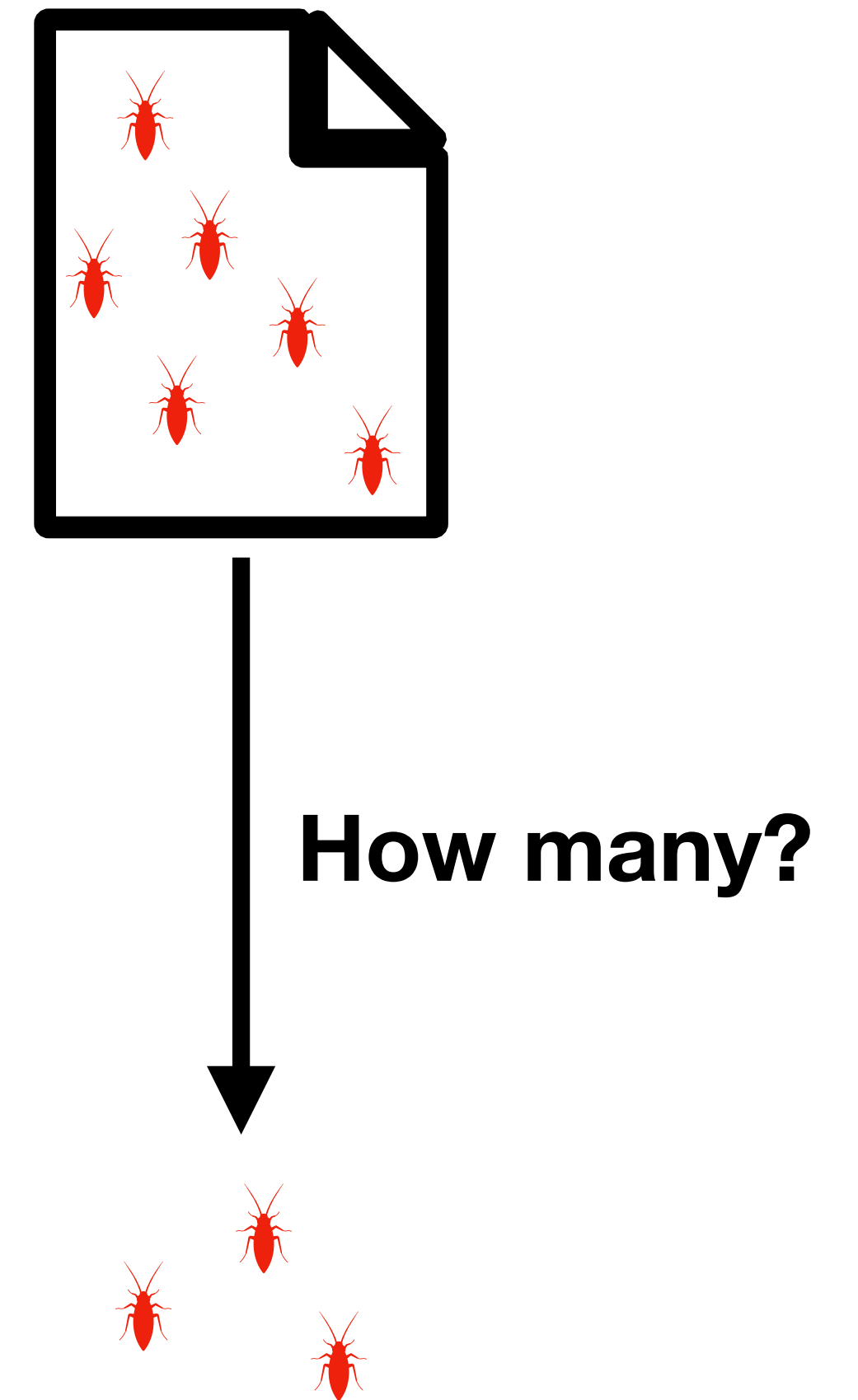
1. Taking a sample:

1. Take a group at random
2. Take a member from the group



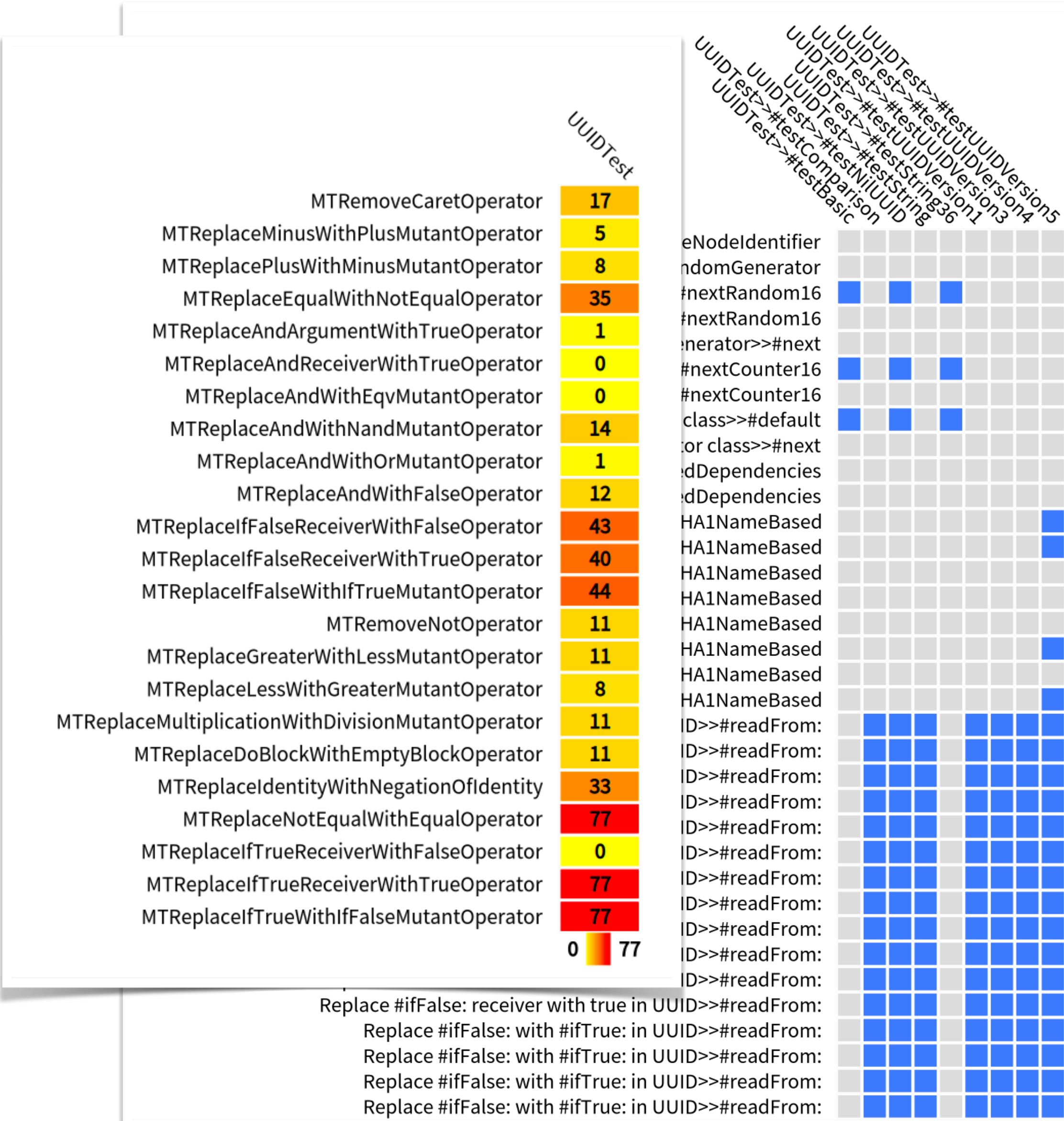
When to Stop Sampling: Budgets!

- Fixed budgets: # or % of mutants
 - Too many: No win in runtime!
 - Too few: Unrealistic mutation score...
- We need a practical solution:
 - **Time budget!**



Mutation Matrix and Heatmap

- **Fine-tune** for your project
- **Advanced** mutation analysis
- Identify
 - **Trivial** mutations: easy to kill
 - **Redundant** tests and mutants
 - Randomization strategy



Possible Extensions and Next Steps

- Select a subset of mutants to run
- Select a subset of tests
- High-order mutants
- Language/Application-specific mutations
- Use mutation testing to evaluate your fuzzer!

Takeaways

- We can automatically assess test case quality
- Mutations introduce artificial bugs and mimic developer issues
- Expensive to run
- Trivial and Equivalent mutants bias results and should be manually inspected

Material

- Mutation Testing Advances: an Analysis and Survey. Advances in Computers Journal. Papadakis et al